

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Компьютерные науки и анализ данных»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Программный проект на тему:
Веб-приложение для группового общения

Выполнил студент:

группы #БКНАД221, 4 курса

Деев Семен Алексеевич

Принял руководитель ВКР:

Промыслов Валентин Валерьевич

Доцент

Факультет компьютерных наук НИУ ВШЭ

Соруководитель:

Овчинников Сергей Андреевич

Ведущий инженер

ООО "ВымпелКом-Информационные технологии"

Содержание

Аннотация	4
1 Введение	5
2 Анализ существующих приложений	7
2.1 Discord	8
2.2 Zoom	8
2.3 Jitsi Meet	9
2.4 Matrix / Element	9
2.5 TeamSpeak	10
Сводная таблица	11
3 Формирование требований к приложению	13
3.1 Функциональные требования	13
3.2 Нефункциональные требования	15
4 Архитектура и проектирование	16
4.1 Общая архитектура	16
4.2 Модель данных	16
4.3 Backend-архитектура	17
4.4 WebSocket и доставка сообщений	17
4.5 WebRTC и голосовые топики	18
4.6 Frontend-архитектура	18
5 Реализация	19
5.1 Реализация backend	19
5.2 Реализация frontend	21
5.3 Локальная инфраструктура	22
6 Тестирование и оценка результата	22
6.1 Подход к тестированию	22
6.2 Сценарии функционального тестирования	23
6.3 Оценка производительности	24
6.4 Оценка соответствия требованиям	24

7 Заключение	25
Список литературы	26

Аннотация

В выпускной квалификационной работе рассматривается проектирование и реализация веб-приложения «Chatterly», предназначенного для группового общения пользователей в рамках онлайн-сообществ. Актуальность работы обусловлена потребностью в доступных браузерных коммуникационных сервисах, которые объединяют постоянные сообщества, тематические каналы, историю переписки и групповые звонки, но не требуют установки отдельного клиента и не обладают избыточной сложностью интерфейса.

Целью работы является разработка прототипа веб-приложения для организации пользовательских сообществ, личной переписки, обмена текстовыми сообщениями в реальном времени и проведения аудио- и видеозвонков. В ходе работы проведён анализ существующих решений, сформированы функциональные и нефункциональные требования, спроектирована архитектура системы и реализованы основные компоненты приложения.

Серверная часть приложения реализована на языке Go, клиентская часть – с использованием JavaScript и SolidJS. Для хранения данных применяется PostgreSQL, для сессий, кэширования и публикации событий – Redis, для работы с пользовательскими изображениями – S3-совместимое хранилище. Обмен событиями в реальном времени реализован на основе WebSocket, а аудио- и видеосвязь – на основе WebRTC.

Результатом работы является прототип коммуникационного веб-приложения, включающий управление пользователями, серверы и топики, личные сообщения, историю переписки, доставку сообщений в реальном времени и базовую поддержку групповых звонков. В работе также рассмотрены ограничения текущей реализации и направления дальнейшего развития системы.

Ключевые слова

Веб-приложение, групповая коммуникация, обмен сообщениями в реальном времени, видеосвязь, Go, JavaScript, PostgreSQL, Redis, WebSocket, WebRTC.

1 Введение

Современные цифровые сообщества в значительной степени опираются на программные средства сетевой коммуникации. Такие средства используются в образовательных группах, профессиональных командах, игровых сообществах, клубах по интересам и иных объединениях, где пользователям необходимо не только обмениваться отдельными сообщениями, но и поддерживать устойчивую структуру общения. Для подобных сценариев важны постоянные пространства взаимодействия, тематическое разделение обсуждений, история переписки, оперативная доставка новых сообщений, а также возможность голосовой и видеосвязи.

Развитие веб-технологий привело к тому, что значительная часть коммуникационных функций может быть реализована непосредственно в браузере. Пользовательский сценарий в этом случае становится проще: для доступа к системе достаточно открыть веб-страницу, пройти регистрацию или авторизацию и перейти к нужному сообществу или диалогу. Такой подход особенно важен для пользователей, которым требуется быстрое подключение к общению без установки отдельного клиента и сложной настройки окружения.

Одним из наиболее близких по назначению решений является Discord, предоставляющий модель серверов, каналов, личных сообщений и голосовых комнат [3]. Однако использование Discord как универсальной основы для целевой аудитории ограничено двумя обстоятельствами. Во-первых, интерфейс и система настроек платформы могут быть избыточными для пользователей, которым необходим базовый набор функций группового общения. Во-вторых, доступ к Discord на территории Российской Федерации ограничен [10]. Следовательно, несмотря на функциональную близость к целевой модели, Discord не устраняет полностью рассматриваемую практическую проблему.

Другие распространённые решения также закрывают только отдельные части задачи. Zoom и Jitsi Meet удобны для видеоконференций и разовых встреч, но не ориентированы на модель постоянных сообществ с тематическими каналами и долговременной историей обсуждений [6, 2]. TeamSpeak хорошо решает задачу голосового общения, однако в меньшей степени соответствует современным требованиям к браузерному интерфейсу, видеосвязи и текстовому взаимодействию [4]. Matrix и Element предоставляют развитую модель независимой коммуникационной инфраструктуры и контроля данных, но могут быть сложнее для массового пользователя с точки зрения настройки и освоения [5, 1]. Таким образом, существующие аналоги либо недоступны, либо сложны, либо ориентированы на более узкий коммуникационный сценарий.

Актуальность настоящей работы определяется необходимостью разработки доступного веб-приложения, которое объединяет базовые возможности современных коммуникационных платформ: создание сообществ, разделение общения по тематическим каналам, личные сообщения, хранение истории, обмен событиями в реальном времени и групповые аудио-/видеозвонки. При этом особое значение имеет простота пользовательского интерфейса, поскольку целевое приложение должно быть пригодно не только для технически подготовленных пользователей, но и для обычных участников онлайн-сообществ.

Объектом работы является веб-приложение для группового общения пользователей. Предметом работы являются архитектурные и программные решения, необходимые для реализации такого приложения: модель данных, серверная бизнес-логика, клиентский интерфейс, механизм доставки событий в реальном времени и механизм установления мультимедийных соединений между участниками звонка.

Целью выпускной квалификационной работы является проектирование и реализация прототипа веб-приложения «Chatterry», предназначенного для организации группового общения пользователей в рамках сообществ, структурированных по тематическим топикам.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 провести анализ существующих коммуникационных приложений и определить их ограничения относительно целевого сценария;
- 2 сформировать функциональные и нефункциональные требования к разрабатываемой системе;
- 3 спроектировать архитектуру серверной части, клиентской части, базы данных, WebSocket-контура и подсистемы аудио-/видеозвонков;
- 4 реализовать серверную часть приложения на языке Go с REST API, WebSocket-интерфейсом, доступом к PostgreSQL, Redis и S3-совместимому хранилищу;
- 5 реализовать клиентскую часть приложения на JavaScript и SolidJS, включая страницы авторизации, личных сообщений, серверов, текстовых и голосовых топиков;
- 6 реализовать обмен текстовыми сообщениями в реальном времени и сохранение истории сообщений;
- 7 реализовать базовую поддержку групповых аудио- и видеозвонков с использованием WebRTC;

8 описать подход к тестированию и оценить соответствие полученного прототипа сформированным требованиям.

Практическая значимость работы состоит в создании прототипа независимого веб-приложения, которое может использоваться как основа для дальнейшего развития коммуникационной платформы. В отличие от решений, ориентированных только на видеовстречи или только на голосовое общение, разрабатываемая система объединяет несколько связанных пользовательских сценариев: постоянные сообщества, тематические топики, личную переписку, историю сообщений и звонки. В отличие от функционально насыщенных коммерческих платформ, Chatterry рассматривается как более простой по пользовательской модели прототип, в котором основное внимание уделяется базовым сценариям группового взаимодействия.

Техническая значимость работы связана с интеграцией нескольких механизмов, характерных для современных приложений реального времени. Серверная часть реализована на языке Go и построена по слоистой архитектуре: выделены доменные модели, сервисы бизнес-логики, адаптеры к внешним хранилищам и HTTP/WebSocket API. Для хранения постоянных данных используется PostgreSQL, для кэширования сессий и публикации событий – Redis, для пользовательских изображений – S3-совместимое объектное хранилище. Доставка событий между пользователями реализована на основе WebSocket, а передача аудио- и видеопотоков – на основе WebRTC.

В результате работы разработан прототип приложения, включающий регистрацию и аутентификацию пользователей, управление профилем, создание серверов и топиков, личные сообщения, текстовые сообщения в топиках, постраничную загрузку истории, уведомления о новых сообщениях и пользовательский интерфейс голосовых топиков. Также реализована серверная обработка событий звонков, включая подключение участников, передачу сигнальных сообщений и обработку медиапотоков.

2 Анализ существующих приложений

План итогового текста раздела:

- использовать анализ аналогов из отчёта по преддипломной практике как основу данного раздела;
- перед описанием отдельных решений объяснить, что цель анализа состоит не в простом перечислении функций, а в обосновании необходимости разработки Chatterry;

- явно перечислить критерии сравнения:
 - наличие постоянных сообществ или серверов;
 - поддержка тематических текстовых каналов;
 - обмен сообщениями в реальном времени и хранение истории;
 - поддержка групповой голосовой связи;
 - поддержка групповой видеосвязи и демонстрации экрана;
 - простота входа для пользователя;
 - возможность самостоятельного развёртывания или контроля данных;
 - доступность для целевой аудитории.
- после каждого аналога писать не только возможности продукта, но и вывод для проектируемой системы: какую идею следует использовать и какое ограничение требуется устранить.

2.1 Discord

- описать Discord как наиболее близкий функциональный аналог: серверы, каналы, личные сообщения, голосовые комнаты, видеосвязь, демонстрация экрана, роли и интеграции [3];
- подчеркнуть сильные стороны: удобная модель постоянных сообществ, разделение общения по каналам, развитые механизмы управления участниками;
- указать ограничения для целевой аудитории: перегруженный интерфейс для новичков, большое количество настроек и ограниченная доступность в Российской Федерации [10];
- сделать вывод для Chatterly: взять модель серверов и каналов как ориентир, но ограничить MVP базовыми сценариями и сделать интерфейс проще.

2.2 Zoom

- описать Zoom как платформу для видеоконференций, онлайн-встреч и вебинаров с поддержкой демонстрации экрана [6];

- отметить сильную сторону: низкий барьер подключения к разовой встрече и устойчивый пользовательский сценарий видеосвязи;
- указать ограничение: Zoom не строится вокруг постоянных сообществ, серверов, тематических каналов и долговременной истории обсуждений;
- упомянуть риск ограниченной доступности для российских пользователей [9];
- сделать вывод для Chatterly: видеосвязь должна быть встроенной возможностью внутри сообщества, а не единственной центральной сущностью приложения.

2.3 Jitsi Meet

- описать Jitsi Meet как открытое браузерное решение для быстрых аудио- и видеовстреч без обязательной установки отдельного клиента [2];
- выделить сильные стороны: простота старта, работа в браузере, пригодность для самостоятельного развёртывания;
- указать ограничения: слабая модель постоянных сообществ, отсутствие развитых профилей, серверов, тематических каналов и полноценной истории общения [8];
- сделать вывод для Chatterly: использовать идею браузерной доступности звонка, но совместить её с постоянными текстовыми пространствами и пользовательскими профилями.

2.4 Matrix / Element

- описать Matrix как децентрализованный протокол, а Element как один из популярных клиентов, поддерживающих сообщения, VoIP и видеосвязь [5, 1];
- выделить сильные стороны: федерация, автономность, контроль хранения данных, ориентация на безопасность [7];
- указать ограничение: архитектура и первичная настройка сложнее для массового пользователя, чем у централизованных сервисов;
- сделать вывод для Chatterly: сохранить идею независимости от конкретной зарубежной платформы, но выбрать более простой централизованный MVP, который легче реализовать и использовать.

2.5 TeamSpeak

- описать TeamSpeak как VoIP-клиент, ориентированный прежде всего на голосовое общение в сообществах [4];
- выделить сильные стороны: низкие задержки, качество аудио, голосовые комнаты и управление доступом;
- указать ограничения: устаревший пользовательский опыт, слабая поддержка современных веб-сценариев, ограниченная видеосвязь и текстовое взаимодействие;
- сделать вывод для Chattery: голосовые комнаты важны, но они должны быть встроены в современное веб-приложение вместе с текстовыми каналами и видеосвязью.

Сводная таблица

Решение	Сообщества	Текстовые каналы	Голос	Видео	Веб-доступ	Что отсутствует или ограничено	Вывод для проекта
Discord	есть	есть	есть	есть	есть	ограниченная доступность в РФ, перегруженность интерфейса	нужен аналог с базовым набором функций и более простой моделью использования
Zoom	нет	нет	есть	есть	есть	нет постоянных сообществ, каналов и истории тематического общения	видеосвязь должна быть дополнением к сообществам, а не единственной функцией
Jitsi Meet	нет	нет	есть	есть	есть	нет развитых профилей, серверов, каналов и постоянной истории сообщений	полезна модель быстрого браузерного звонка, но нужны текстовые чаты
Matrix / Element	есть	есть	есть	есть	есть	высокая сложность настройки и освоения для массового пользователя	следует сохранить идею контроля данных, но упростить пользовательский сценарий
TeamSpeak	частично	нет	есть	ограничено	ограничено	устаревший интерфейс, слабая поддержка видео и текстового взаимодействия	голосовые комнаты важны, но должны быть встроены в современное приложение

Таблица 2.1: Сравнение существующих решений по функциям, значимым для проекта

План вывода по разделу:

- сослаться на Таблицу 2.1;
- написать, что Discord задаёт наиболее близкую функциональную модель, но не подходит как единственная основа из-за доступности и сложности;
- написать, что Zoom и Jitsi Meet решают задачу видеовстреч, но не задачу постоянного сообщества;
- написать, что TeamSpeak подтверждает значимость голосовых комнат, но не покрывает современные веб-сценарии;
- написать, что Matrix / Element важны как пример автономной коммуникационной инфраструктуры, но требуют упрощения UX для массовой аудитории;

- сформулировать итог: собственное приложение должно объединить сообщества, топик-ки, личные сообщения, историю, WebSocket-события и WebRTC-звонки.

3 Формирование требований к приложению

План вводного текста раздела:

- написать, что требования формируются на основе анализа аналогов и целей MVP;
- объяснить, что требования делятся на функциональные, описывающие поведение системы, и нефункциональные, описывающие качественные характеристики;
- перед перечнем требований добавить таблицу прослеживаемости, чтобы показать связь между недостатками аналогов и решениями в Chatterly.

Выявленное ограничение	Где проявляется	Требование к Chatterly
Нет единой структуры постоянных сообществ	Zoom, Jitsi Meet	реализовать серверы/сообщества как основные пространства взаимодействия
Нет тематических каналов внутри сообщества	Zoom, Jitsi Meet, TeamSpeak	реализовать текстовые и голосовые топики внутри каждого сообщества
Сервис ориентирован только на видеовстречи или голосовую связь	Zoom, Jitsi Meet, TeamSpeak	совместить текстовое общение, голосовые и видеозвонки в одном веб-приложении
Недостаточная доступность для целевой аудитории	Discord, потенциально Zoom	обеспечить независимую реализацию, не зависящую от конкретной зарубежной платформы
Сложность настройки и освоения	Discord, Matrix / Element, TeamSpeak	сделать пользовательский интерфейс простым и сократить число действий для основных сценариев
Необходимость обмена событиями в реальном времени	все решения, поддерживающие чаты и звонки	использовать WebSocket для сообщений и событий звонков
Необходимость браузерной аудио- и видеосвязи	Discord, Zoom, Jitsi Meet, Matrix / Element	использовать WebRTC и реализовать серверный signaling
Необходимость сохранения истории общения	Discord, Matrix / Element	хранить сообщения в базе данных и поддерживать постраничную загрузку истории

Таблица 3.1: Прослеживаемость требований от анализа аналогов

3.1 Функциональные требования

План текста подраздела:

- перед списком сослаться на Таблицу 3.1;
- пояснить, что в итоговом тексте каждое требование должно быть описано как пользовательская возможность и как поведение системы.

- 1 **Регистрация и аутентификация пользователей.** Описать создание учётной записи по логину, имени пользователя и паролю; вход; выход; получение текущего пользователя через защищённый endpoint; хранение сессии в cookie и Redis.
- 2 **Управление пользовательским профилем.** Описать просмотр текущего профиля, изменение имени, логина и пароля, удаление профиля, загрузку и удаление аватара, генерацию identicon при отсутствии пользовательского изображения.
- 3 **Личные сообщения.** Описать поиск пользователей, с которыми ещё нет личного диалога, создание ДМ, список диалогов, хранение последнего сообщения, признак непрочитанности и ручку отметки сообщения прочитанным.
- 4 **Сообщества и участники.** Описать создание сервера, вступление в сервер, выход из сервера, удаление сервера, роль владельца и роль участника, а также проверку прав при изменении структуры сервера.
- 5 **Тематические топики.** Описать создание, редактирование и удаление топики двух типов: текстовый топик для сообщений и голосовой топик для звонков.
- 6 **Текстовые сообщения в реальном времени.** Описать отправку сообщений через REST endpoint, сохранение в PostgreSQL и доставку события всем участникам через Redis pub/sub и WebSocket.
- 7 **История сообщений и пагинация.** Описать загрузку первой страницы сообщений и следующих страниц по cursor, состоящему из идентификатора чата, времени создания и идентификатора последнего загруженного сообщения.
- 8 **WebSocket-подписки.** Описать подключение к единому endpoint /ws/, события join, leave, ping, pong, message, error, а также типы каналов dm, text_topic, voice_topic.
- 9 **Голосовые и видеозвонки.** Описать подключение к голосовому топику, получение списка участников, передачу микрофона, камеры и демонстрации экрана через WebRTC.
- 10 **WebRTC signaling.** Описать обмен событиями voice_offer, voice_answer, voice_ice_candidate, voice_ice_candidates, voice_state, voice_joined, voice_left.
- 11 **Пользовательский интерфейс чата.** Описать список диалогов, список серверов, переходы между топиками, автопрокрутку новых сообщений, догрузку старых сообщений при прокрутке вверх и toast-уведомления о входящих личных сообщениях.

- 12 **Пользовательский интерфейс звонка.** Описать локальный preview, плитки удалённых участников, кнопки включения микрофона, камеры и демонстрации экрана, выбор устройств и отображение ошибок доступа к media devices.

3.2 Нефункциональные требования

План текста подраздела:

- 1 **Доступность через браузер.** Описать, что пользователь должен работать с приложением без установки отдельного клиента; это особенно важно в сравнении с TeamSpeak и другими desktop-first решениями.
- 2 **Простота интерфейса.** Описать, что основные сценарии должны выполняться с небольшим числом действий: регистрация, вход, поиск пользователя, создание DM, создание сервера, переход в топик, отправка сообщения, подключение к звонку.
- 3 **Сохранность данных.** Описать, что пользователи, серверы, участники, топики, личные сообщения и сообщения топиков хранятся в PostgreSQL.
- 4 **Производительность.** Зафиксировать целевое требование: не менее десяти активных пользователей в одном чате или звонке без заметной пользовательской задержки; в тестовом разделе определить, как это проверять.
- 5 **Масштабируемость real-time контура.** Описать использование Redis pub/sub для доставки событий пользователю и возможность запуска нескольких backend-узлов для WebSocket и voice topic ownership.
- 6 **Безопасность.** Описать bcrypt-хэширование паролей, дополнительную соль на основе логина, HttpOnly cookie, SameSite Lax, проверку доступа к DM и топикам на backend-стороне.
- 7 **Поддерживаемость.** Описать разделение на слои: domain, adapters, services, API handlers, stores, frontend features; подчеркнуть, что бизнес-логика не размещается в cmd/main.go.
- 8 **Наблюдаемость.** Описать имеющееся логирование ошибок и HTTP request logging, а сбор метрик ошибок, времени ответа и активных соединений указать как направление развития, если он не будет реализован к финальной версии.

4 Архитектура и проектирование

4.1 Общая архитектура

- описать Chatterry как клиент-серверное веб-приложение;
- указать основные внешние компоненты: браузер пользователя, Go backend, PostgreSQL, Redis, S3-совместимое хранилище, WebRTC/STUN-инфраструктура;
- объяснить поток обычного HTTP-запроса: frontend вызывает REST endpoint, API handler валидирует запрос, service выполняет бизнес-логику, adapter обращается к PostgreSQL, результат возвращается клиенту;
- объяснить поток real-time события: сообщение сохраняется в БД, backend публикует событие в Redis-канал пользователя, WebSocket session получает событие и отправляет его в браузер;
- указать, что `cmd/main.go` является composition root: создаёт конфигурацию, logger, подключения к Postgres/Redis/S3, transaction manager, adapters, stores, services, syncers, WebSocket manager и HTTP server.

4.2 Модель данных

- описать таблицу `users`: идентификатор, username, login, hashed password, avatar id, даты создания и обновления;
- описать таблицы личных сообщений: `dms`, `dm_participants`, `dm_messages`; объяснить, что `last_message_id` и `last_read_message_id` нужны для превью диалога и непрочитанных сообщений;
- описать таблицы сообществ: `servers`, `server_participants`, `topics`, `topic_messages`;
- объяснить роли `owner` и `member`; владелец создаёт и изменяет структуру сервера, участник получает доступ к топикам;
- описать ограничения уникальности: уникальные логины и имена пользователей, уникальный участник в DM или server, уникальное имя топика одного типа внутри сервера;
- описать индексы сообщений DM по `dm_id`, `created_at` и `id`, так как история загружается постранично в обратном хронологическом порядке;

- в финальной версии добавить ER-диаграмму или схему таблиц, если её можно подготовить без перегрузки текста.

4.3 Backend-архитектура

- описать слой `internal/domain`: доменные структуры `User`, `DM`, `Server`, `Topic`, `TopicMessage`, `typed identifiers` и `value object Password`;
- описать слой `internal/api`: HTTP handlers для `user`, `image`, `dm`, `server`, `websocket` и `web`;
- описать слой `internal/service`: бизнес-логика пользователей, DM, серверов, текстовых топиков, изображений, WebSocket sessions и voice topics;
- описать слой `internal/adapter`: адаптеры PostgreSQL, Redis и S3, скрывающие детали внешних хранилищ от сервисов;
- описать `internal/store`: периодически синхронизируемые in-memory кэши пользователей, DM и серверов, используемые для быстрых lookup и поиска;
- описать `transaction manager`: операции создания DM, сообщений, серверов и топиков выполняются в транзакциях, чтобы связанные изменения сохранялись согласованно;
- объяснить, почему выбран Go: простая модель конкурентности, хорошая стандартная библиотека для HTTP, удобство построения сетевых сервисов;
- объяснить, почему выбран `chi`: лёгкий router и middleware для request id, recovery, heartbeat и request logging.

4.4 WebSocket и доставка сообщений

- описать endpoint `/ws/`, доступный только после аутентификации;
- описать жизненный цикл соединения: ассепт, создание `Connection`, регистрация в пользовательской session, запуск read pump и write pump, cleanup при закрытии;
- описать heartbeat: backend отправляет `ping`, frontend отвечает `pong`, отсутствие ответа приводит к закрытию соединения;
- описать подписку на каналы: клиент отправляет `join` с каналом, backend проверяет доступ к DM, text topic или voice topic;

- описать доставку сообщений: сервис DM или text topic публикует `message` в Redis-каналы всех участников, WebSocket manager доставляет событие активным соединениям пользователя;
- описать поведение frontend: singleton WebSocket store поддерживает переподключение, повторно отправляет `join` для активных каналов и буферизует события при временном отсутствии соединения.

4.5 WebRTC и голосовые топики

- описать назначение voice topic service: управление комнатами звонков внутри голосовых топиков;
- описать выбор владельца комнаты через Redis key `voice:topic:<id>:owner`; если несколько backend-узлов обслуживают систему, события `join/leave/signal` перенаправляются владельцу комнаты;
- описать хранение активных участников через Redis sorted set `voice:topic:<id>:participant` и периодическое обновление TTL;
- описать signaling-события: `offer`, `answer`, ICE candidate и batch ICE candidates;
- описать использование библиотеки Pion WebRTC на backend: создание peer connection, регистрация кодеков, STUN server, настройка публичного IP и диапазона UDP-портов;
- описать forwarding медиапоток: backend принимает remote track от участника, создаёт local track и добавляет его другим участникам комнаты;
- описать renegotiation при появлении или исчезновении треков, а также запрос ключевых кадров для восстановления видео;
- описать ограничения: без TURN server соединения могут быть нестабильны за строгими NAT; TURN следует вынести в перспективы развития.

4.6 Frontend-архитектура

- описать клиентскую часть как SolidJS-приложение на Vite;
- описать маршруты приложения: `/app/dm`, `/app/dm/search`, `/app/dm/:dmId`, `/app/server`, `/app/server/create`, `/app/server/manage`, `/app/server/:serverId/text/:topicId`, `/app/server/:serverId/edit`;

- описать разделение frontend по features: auth, dm, server, chat, voice;
- описать общий API-клиент: обработка JSON, сетевых ошибок, ошибок авторизации и ошибок backend;
- описать общий WebSocket store: единое соединение, подписчики событий, message handlers, error handlers, active channel map и pending events;
- описать chat component: загрузка первой страницы, догрузка истории при скролле вверх, дедупликация сообщений, автопрокрутка вниз, отправка сообщений;
- описать voice UI: local tile, remote tiles, статус звонка, меню микрофона/камеры/экрана, настройки media devices.

5 Реализация

5.1 Реализация backend

- описать реализацию конфигурации через переменные окружения: HTTP host/port, Postgres URL, Redis address, S3 endpoint, session parameters, chat message limit, image limits, voice settings;
- описать пользовательский сервис:
 - создание пользователя с bcrypt-хэшированием пароля;
 - вход по login/password;
 - создание сессии и запись session id в Redis;
 - middleware, которая читает cookie, проверяет Redis и помещает user id в request context;
 - обновление профиля с проверкой текущего пароля при изменении login или password;
- описать image service:
 - получение аватара по username;
 - генерация identicon, если пользователь не загрузил изображение;
 - проверка размера и разрешения загружаемого изображения;
 - конвертация изображения в JPEG и сохранение в S3-совместимое хранилище;

- описать DM service:
 - поиск пользователей без существующего DM;
 - создание DM с двумя участниками;
 - отправка сообщения, обновление последнего сообщения и отметка сообщения отправителя как прочитанного;
 - загрузка первой и следующих страниц истории;
 - проверка доступа участника перед чтением и подпиской;
- описать server service:
 - создание сервера и автоматическое добавление создателя как owner;
 - вступление и выход из сервера;
 - обновление и удаление сервера;
 - создание, обновление и удаление текстовых и голосовых топиков;
 - проверка роли owner при изменении структуры сервера;
- описать text topic service:
 - проверка, что отправитель состоит в сервере;
 - проверка, что топик имеет тип text;
 - сохранение сообщения в PostgreSQL;
 - публикация WebSocket-события всем участникам сервера;
- описать WebSocket manager:
 - пользовательские sessions и несколько соединений одного пользователя;
 - Redis subscription на канал пользователя;
 - read pump для входящих событий;
 - write pump для исходящих событий и heartbeat;
 - cleanup с автоматическим выходом из voice topic;
- описать voice topic service:
 - join/leave пользователя в голосовой топик;

- отправка `voice_state` новому участнику;
- broadcast `voice_joined` и `voice_left`;
- обработка `offer`, `answer` и ICE candidates;
- forwarding RTP-треков между участниками;
- закрытие `peer connection` и удаление треков при выходе участника.

5.2 Реализация frontend

- описать страницы входа и регистрации: отдельные entypoints для `login.html` и `signup.html`, вызовы API создания пользователя и входа;
- описать главный app shell: router с base path `/app`, провайдеры, глобальные DM-уведомления и toast storage;
- описать DM UI:
 - список личных диалогов в sidebar;
 - поиск пользователей;
 - создание DM из результата поиска;
 - обновление preview и unread состояния при входящем WebSocket-сообщении;
- описать server UI:
 - список серверов пользователя;
 - создание сервера;
 - поиск и присоединение к серверам;
 - управление сервером и топиками;
 - переходы в text topic и voice topic;
- описать chat UI:
 - единый компонент для DM и text topic;
 - REST-загрузка истории;
 - подписка на WebSocket-канал при открытии чата;
 - отправка сообщений через REST API;

- обработка live-сообщений и догрузка старых сообщений;
- описать voice UI:
 - автоматический join в voice topic при открытии страницы;
 - создание `RTCPeerConnection`;
 - синхронизация local tracks при включении микрофона, камеры или экрана;
 - обработка server offer/answer/ICE;
 - отображение удалённых участников и локального preview;
 - выбор камеры, микрофона и speaker device, если браузер поддерживает `setSinkId`.
- отдельно описать текущие ограничения frontend: profile settings modal не должен быть заявлен как полностью завершённый, если к финальной версии не подключены реальные endpoint-ы обновления профиля и аватара.

5.3 Локальная инфраструктура

- описать `docker-compose.local.yaml`: PostgreSQL, Redis, MinIO и вспомогательный контейнер создания bucket;
- описать Makefile-команды: `make up`, `make run`, `make run-web`, `make build`, `make build-web`, `make test`;
- описать миграции Goose и генерацию SQLC-кода из `queries/*.sql` и `migrations/*.sql`;
- указать, что production deploy находится вне текущего реализованного MVP, если не будет завершён до сдачи.

6 Тестирование и оценка результата

6.1 Подход к тестированию

- написать, что для программного проекта требуется не только описать реализацию, но и показать корректность ключевых пользовательских сценариев;
- разделить проверки на несколько уровней: сборка, backend API, real-time события, frontend сценарии, звонки, безопасность доступа, производительность;

- честно указать состояние автотестов: в репозитории есть команда `go test ./...`, но `unit/e2e` тесты для сервисов и `API handlers` отмечены как незавершённые; если к финалу они будут добавлены, заменить этот пункт на описание конкретных тестов;
- привести таблицу сценариев тестирования, где для каждого сценария указать входные данные, ожидаемый результат и фактический статус.

6.2 Сценарии функционального тестирования

- **Регистрация и вход.** Создать пользователя, войти с правильным паролем, убедиться, что `/v1/user/me` возвращает текущий профиль; проверить ошибку при неверном пароле.
- **Сессии.** Проверить, что защищённые endpoints возвращают ошибку без cookie и работают после входа; проверить `logout` и невозможность повторного доступа по очищенной сессии.
- **Профиль и аватар.** Проверить получение `identicon` для пользователя без аватара, загрузку изображения допустимого размера, отказ при слишком большом файле или некорректном формате.
- **Личные сообщения.** Создать двух пользователей, найти второго пользователя, создать DM, отправить сообщение, проверить появление сообщения у обоих пользователей и обновление `preview`.
- **История DM.** Создать больше сообщений, чем `MESSAGES_LIMIT`, загрузить первую страницу и следующую страницу по `cursor`, проверить порядок и отсутствие дублей.
- **Серверы и топики.** Создать сервер, убедиться, что создатель получил роль `owner`, создать `text topic` и `voice topic`, изменить и удалить топик.
- **Права доступа.** Проверить, что пользователь, не состоящий в сервере, не может читать сообщения топика и подписываться на WebSocket-канал этого топика; проверить, что `member` не может изменять структуру сервера, если такое ограничение реализовано.
- **Текстовый топик.** Отправить сообщение в `text topic` и проверить его доставку всем участникам сервера через WebSocket.
- **WebSocket reconnect.** Разорвать соединение, дождаться переподключения `frontend`, проверить повторный `join` активного канала и получение новых сообщений.

- **Голосовой топик.** Открыть voice topic двумя пользователями, проверить voice_state, отображение участников, передачу микрофона, камеры и отключение участника при закрытии вкладки.
- **Демонстрация экрана.** Включить screen share, проверить появление видеопотока у другого участника и корректное прекращение трансляции.

6.3 Оценка производительности

- описать целевую метрику для чата: задержка от отправки сообщения одним пользователем до отображения у других участников;
- описать целевую метрику для WebSocket: число активных соединений, стабильность heartbeat, отсутствие массовых reconnect при десяти пользователях;
- описать целевую метрику для звонка: успешность подключения, время до появления удалённого участника, устойчивость аудио/видео при двух и более участниках;
- зафиксировать тестовую конфигурацию: машина, браузер, число пользователей, способ эмуляции пользователей, параметры PostgreSQL/Redis/backend;
- если нагрузочное тестирование не будет проведено, написать это как ограничение и оставить только ручную проверку сценариев MVP.

6.4 Оценка соответствия требованиям

- составить таблицу соответствия: требование, реализованный компонент, способ проверки, статус;
- для реализованных требований ссылаться на разделы реализации: user service, DM service, server service, text topic service, WebSocket manager, voice topic service, frontend routes;
- для частично реализованных требований указать причину и план завершения, например profile settings modal, автотесты, метрики, production deploy, TURN server;
- в выводе раздела написать, какие сценарии MVP подтверждены, а какие остаются перспективами развития.

7 Заключение

План итогового текста заключения:

- вернуться к цели из введения и написать, что в работе был спроектирован и реализован MVP веб-приложения Chattery для группового общения;
- перечислить результаты:
 - проведён анализ Discord, Zoom, Jitsi Meet, Matrix / Element и TeamSpeak;
 - сформированы функциональные и нефункциональные требования;
 - спроектирована архитектура backend, frontend, хранилищ, WebSocket и WebRTC;
 - реализованы пользователи, сессии, профили, аватары, серверы, топики, личные сообщения, история сообщений и доставка событий в реальном времени;
 - реализованы голосовые топики с WebRTC signaling, локальными media controls и отображением участников;
 - подготовлена локальная инфраструктура разработки на Docker Compose;
- описать практическую полезность: приложение демонстрирует независимую браузерную платформу для сообществ, объединяющую текстовое общение и звонки;
- указать ограничения честно и конкретно: отсутствие production deploy, неполное автотестирование, необходимость TURN server для сложных сетей, необходимость расширить роли и модерацию, необходимость подключить или доработать все сценарии профиля, если они не будут завершены;
- описать направления дальнейшего развития: e2e и unit tests, мониторинг и метрики, нагрузочное тестирование, мобильная адаптация, расширенные права доступа, push-уведомления, файлы в чатах, TURN-инфраструктура.

Список литературы

- [1] *About Element*. Element. URL: <https://element.io/about> (дата обр. 21.04.2026).
- [2] *About Jitsi Meet*. Jitsi. URL: <https://jitsi.org/jitsi-meet/> (дата обр. 19.04.2026).
- [3] *Discord Server Setup Guide*. Discord. 2 июля 2025. URL: <https://support.discord.com/hc/en-us/articles/33023827550359-Discord-Server-Setup-Guide> (дата обр. 20.04.2026).
- [4] *How to voice chat*. TeamSpeak. 14 апр. 2026. URL: <https://support.teamspeak.com/hc/en-us/articles/35367762165405-How-to-voice-chat> (дата обр. 21.04.2026).
- [5] *Matrix Specification*. Matrix.org. URL: <https://spec.matrix.org/> (дата обр. 21.04.2026).
- [6] *Screen sharing software*. Zoom. URL: <https://www.zoom.com/en/products/virtual-meetings/features/screen-sharing/> (дата обр. 19.04.2026).
- [7] *Secure collaboration platform for enterprises*. Element. URL: <https://element.io/solutions/secure-collaboration> (дата обр. 21.04.2026).
- [8] *Terms & Conditions*. 8x8. URL: <https://jitsi.org/meet-jit-si-terms-of-service/>.
- [9] Евгений Одинцов. *В России предрекли скорую блокировку Zoom*. Газета.Ру. 10 сент. 2025. URL: <https://www.gazeta.ru/social/news/2025/09/10/26692490.shtml> (дата обр. 21.04.2026).
- [10] *Роскомнадзор заблокировал Discord*. РБК. 8 окт. 2024. URL: https://www.rbc.ru/technology_and_media/08/10/2024/67054cbf9a79474670135b84 (дата обр. 22.04.2026).